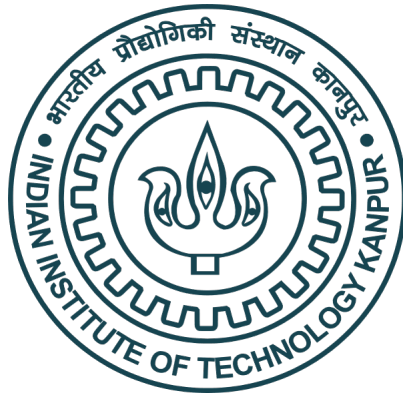

Tries without Trails: Oblivious Data Structure with Distributed ORAM

*A thesis submitted in fulfilment of the requirements
for the degree of Master of Science (by Research)*

by

Lokesh Valeti

Roll No. 231110406



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KANPUR

May 2025

Certificate

It is certified that the work contained in this thesis entitled “**Tries without Trails: Oblivious Data Structure with Distributed ORAM**” by **Lokesh Valeti** has been carried out under my supervision and that it has not been submitted elsewhere for a degree.

Prof. Adithya Vadapalli

Assistant Professor

CSE Department

Indian Institute of Technology Kanpur

May 2025

Declaration

This is to certify that the thesis titled “**Tries without Trails: Oblivious Data Structure with Distributed ORAM**” has been authored by me. It presents the research conducted by me under the supervision of **Prof. Adithya Vadapalli**.

To the best of my knowledge, it is an original work, both in terms of research content and narrative, and has not been submitted elsewhere, in part or in full, for a degree. Further, due credit has been attributed to the relevant state-of-the-art and collaboration (if any) with appropriate citations and acknowledgement, in line with established and practices.

Lokesh Valeti

Roll No. 22111052

CSE Department

Indian Institute of Technology Kanpur

Abstract

Name of the student: **Lokesh Valeti**

Roll No: **231110406**

Degree for which submitted: **MSR**

Department: **CSE Department**

Thesis title: **Tries without Trails: Oblivious Data Structure with Distributed ORAM**

Thesis supervisor: **Prof. Adithya Vadapalli**

Month and year of thesis submission: **May 2025**

In applications such as auto-completion systems and spell checkers, user queries are typically processed through servers that can observe access patterns and infer sensitive information. To address this privacy concern, this thesis focuses on hiding access patterns by designing an oblivious data structure. Specifically, we propose an oblivious Trie data structure that enables secure index-based access while preserving user privacy.

In this work, we have designed a novel memory layout structure that ensures efficient utilization of memory without any memory wastage. Based on this layout, we build two variants of Tries — a basic oblivious Trie and an optimized oblivious Trie — both supporting insert, search, and delete operations. we demonstrate that the optimized Trie significantly outperforms the basic version in terms of communication overhead and computational efficiency. This work lays the foundation for building privacy-preserving applications where the server remains oblivious to the user's search queries, thereby enhancing user confidentiality in sensitive data retrieval scenarios.

Acknowledgements

I would like to sincerely thank my thesis advisor, Prof. Adithya Vadapalli, for his great support, guidance, and encouragement throughout my research. His knowledge, patience, and helpful advice were very important in shaping my work and improving its results. I am truly grateful for his support, which played a big role in helping me complete this thesis successfully.

I am especially grateful to Prof. Adithya Vadapalli for being extremely supportive and understanding, particularly during the difficult phases of my research. His approachable nature, positive attitude, and willingness to help made a significant difference in my journey. He was not just a guide but often like an elder brother, offering encouragement whenever I felt demotivated and helping me regain confidence. His constant support and friendly guidance created a motivating environment that helped me complete my work with renewed energy and focus.

My heartfelt thanks also go to the distinguished members of my thesis defense committee — Prof. Urbi Chatterjee, Prof. Sayak Ray Chowdhury, and Prof. Tushar Sandhan — for their valuable feedback, expert advice, and thoughtful suggestions. Their time, encouragement, and scholarly input have added immense value to this work, and I am genuinely grateful for their support during the evaluation process.

Contents

Acknowledgements	v
List of Figures	viii
1 Introduction	1
2 Background	3
2.1 Secret Sharing	3
2.1.1 Secret Shares of a 1-hot vector	4
2.2 Distributed Point Functions (DPFs)	4
2.3 DPF Construction	7
2.3.1 Incremental DPFs	9
2.4 Secure Multi-party Computation	9
(AND Tuples):	11
(Dot-Product Tuples):	11
2.5 MPC Protocol for Bitwise AND	12
2.6 MPC Protocol for Dot Product	12
2.7 MPC Protocol for Arithmetic Multiplication	13
2.8 Distributed Oblivious RAMs (DORAMs)	14
2.8.1 Floram and Duoram	15
3 PRAC	16
3.1 DORAM operations in PRAC	17
3.1.1 3P-DUORAM Read Protocol	18
3.1.2 3P-DUORAM Update Protocol	19
3.2 Oblivious Data Structures in PRAC	20
4 Oblivious Tries	21
4.1 Memory Layout	21
4.1.1 Tree-Based Representation of a Dictionary	22
4.1.2 Flat Bit Array Representation	24
4.2 Oblivious Tries	27

4.2.1	Basic Implementations	28
4.2.1.1	Oblivious Insert	28
4.2.1.2	Oblivious Search	28
4.2.1.3	Oblivious Delete	29
4.2.2	Optimizations	29
4.2.2.1	Optimized Insert	31
4.2.2.2	Optimized Search	31
4.2.2.3	Optimized Oblivious Delete	33
5	Evaluation	34
5.1	Theoretical Analysis	34
5.2	Experimental Analysis	35
6	Conclusion	40
	Bibliography	42

List of Figures

2.1	Binary tree depiction of a point function where the target value M is located at index $i^* = 0b010 = 2$	5
2.2	The two trees illustrate the DPF structures maintained by P_0 and P_1 . Each node contains a tuple $(s^{(x)}, b)$, where $s^{(x)}$ is the label and b denotes the advice bit. Child nodes are computed using a length-doubling PRG along with correction terms. Leaf nodes lie in the domain $\{0, 1\}^{\lambda \times w}$, and all corresponding nodes are identical across the two trees except along the designated path to the target leaf.	9
2.3	Secure Multi-Party Computation	10
3.1	DORAM Read and Update operations	18
4.1	Flat Array Representation of Trie.	27
4.2	Optimized Trie Representation	31
5.1	Comparing communication cost of Oblivious Tries insert ■ Basic vs. ■ Optimized	36
5.2	Comparing wall clock time of Oblivious Tries insert ■ Basic vs. ■ Optimized	36
5.3	Comparing online Phase communication cost of Oblivious Tries insert ■ Basic vs. ■ Optimized	37
5.4	Comparing communication cost of Oblivious Tries search ■ Basic vs. ■ Optimized	37
5.5	Comparing wall clock time of Oblivious Tries search ■ Basic vs. ■ Optimized	38
5.6	Comparing online Phase communication cost of Oblivious Tries search ■ Basic vs. ■ Optimized	38
5.7	Comparing communication cost of Oblivious Tries delete ■ Basic vs. ■ Optimized	38
5.8	Comparing wall clock time of Oblivious Tries delete ■ Basic vs. ■ Optimized	39

Chapter 1

Introduction

Introduction

In the realm of cybersecurity, when we store data in a database, we typically encrypt it to keep it safe. However, even if the contents are hidden, accessing the data can still leak sensitive information. For instance, if someone observes which parts of the database are accessed frequently (such as specific indices), they might infer what is important or what a user is searching for. To protect against this, a technique called **Oblivious RAM (ORAM)** is used, which hides not only the data but also the access patterns.

Imagine searching for something on Google, like “IIT.” You might get suggestions like “IIT Kanpur,” “IIT Bombay,” and so on. These suggestions are powered by a data structure called a **Trie** — a tree-like structure used to store and search words efficiently, especially when many words share common prefixes. Each node in a Trie represents a single character, and by linking characters from the root, complete words can be formed. Tries allow fast word searches, autocomplete, and prefix-based queries while saving space by sharing common prefixes.

To keep such search indices private in a privacy-preserving system, the database can be split into parts and distributed across multiple servers, where each server only holds a portion of the data and cannot communicate with others. When a user searches for “IIT,” the query is also split and sent across the servers. After performing local computations, the servers send back pieces of the result without learning the

actual search term. This way, user privacy is preserved, and even the servers (or the search engine) do not know what was searched.

Motivated by this, we have developed an **Oblivious Trie** data structure, which enables functionalities like auto-completion systems and word checking while ensuring privacy. We first designed a memory layout for the data structure that stores information very efficiently, without wasting memory space. We then implemented three basic operations — **search**, **insert**, and **delete** — all of which are privacy-preserving, making the servers completely oblivious to the data and access patterns. Furthermore, we introduced an **optimized version** of the data structure to enhance its **communication bandwidth**. We formally prove the correctness of our optimization and perform experiments measuring communication and computation costs across different database sizes.

Chapter 2

Background

This chapter gives a brief outline of the background required to follow the rest of the thesis.

2.1 Secret Sharing

Secret sharing is a cryptographic technique that allows us to distribute a *secret* among two or more parties. There is a parameter t called threshold associated with every secret sharing mechanism. A secret sharing mechanism has two main properties, namely,

1. Any $< t$ -sized subset of the parties will not be able to reconstruct the secret
2. Any subset of the parties of size $\geq t$ can reconstruct the secret.

More formally, we call an (n, t) secret sharing scheme enables a dealer to divide a secret S into n shares. Here t is the threshold parameter that represents the minimum number of shareholders that are required to collude to learn the secret.

This discussion focuses on the $(2, 2)$ scheme, which mandates that both parties cooperate to reconstruct the secret. In this thesis, we utilize three types of shares: additive, XOR, and Boolean shares. These shares can be defined as follows:

- **Additive Shares (Modulo 2^r):** An integer R is additively shared under modulo 2^r . The shares R_0^{AS} and R_1^{AS} , held by parties P_0 and P_1 respectively,

fulfill the condition $R_0^{AS} + R_1^{AS} \equiv R \pmod{2^r}$, with r set to 64. This is referred to as R^{AS} .

- **XOR Shares:** An integer R is shared using the XOR operation. The shares R_0^{XS} and R_1^{XS} , held by parties P_0 and P_1 respectively, satisfy the equation $R_0^{XS} \oplus R_1^{XS} = R$, where $\oplus$ denotes the bitwise XOR operation, and the bit-length is typically $r = 64$. This is denoted as R^{XS} .
- **Single-bit Boolean Shares:** A single bit b is shared using Boolean methods. The shares b_0^{BS} and b_1^{BS} , held by parties P_0 and P_1 respectively, meet the condition $b_0^{BS} \oplus b_1^{BS} = b$. This is indicated as b^{BS} . When a value is Boolean-shared, it represents a single bit; otherwise, it is expressed as an r -bit word.

2.1.1 Secret Shares of a 1-hot vector

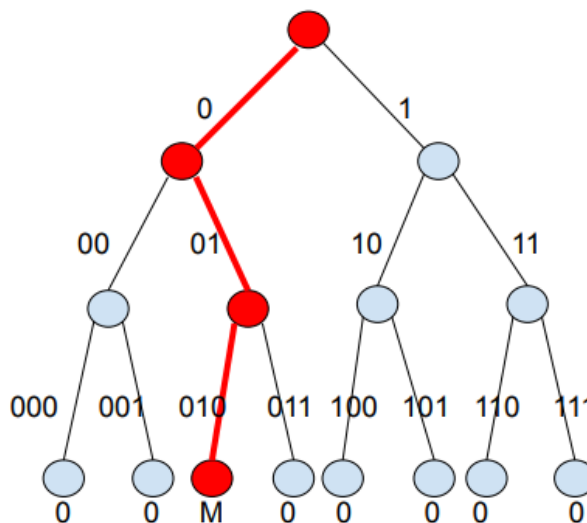
Another way to secret share a value among parties is via so-called 1-hot vectors. For instance, if we want to share a value i^* among two parties, we could choose vectors of size n such that they reconstruct to 0 at all the indices, except at the i^{th} index where they reconstruct to a non-zero value. Secret shares of a 1-hot vector are extremely useful. For instance, the shares of these vectors can be used to private information retrieval (and similarly private writing)[1, 2, 3, 4]. In order to do PIR, we have two parties, called P_0 and P_1 , who hold the replicates of the database, D . The two parties receive the shares of the 1-hot vector, namely, $\vec{r}_0$ and $\vec{r}_1$, with the property that, $\vec{r}_0 + \vec{r}_1 = e_i$. The parties then compute, $\langle D, \vec{r}_0 \rangle$ and $\langle D, \vec{r}_1 \rangle$ respectively and send them to the client. The client can then compute, $D[i] \leftarrow \langle D, \vec{r}_0 \rangle + \langle D, \vec{r}_1 \rangle$. While the aforementioned protocol works, the problem is that, the communication cost is size of the database. For instance, if we want to privately download a tweet, we would have to send download requests of the size of the Twitter database.

2.2 Distributed Point Functions (DPFs)

Distributed Point Functions are a cryptographic technique provides a solution for the aforementioned problem. More formally, an (n, t) -DPF is a cryptographic technique that allows us to algorithmically share a 1-hot vector among n parties such that any subset of size of parties $\geq t$ parties can reconstruct the secret.

Definition 2.1. The function $f_{\alpha,\beta}$ outputs a value β at a secret point α , and zero everywhere else. That is,

Over the entire domain, this function forms a one-hot vector, meaning it is zero at all points except for position α , where it is β .



Point functions can be seen as binary trees. In this representation, each leaf corresponds to an input in the domain. The red path represents the target point i^* , where the function evaluates to the target value M . All other paths lead to leaves that yield a value of 0. All the input value evaluations forms a one-hot vector. Sharing a full 1-hot vector for a domain of size N requires $\mathcal{O}(N)$ communication.

which is inefficient. Modern DPFs solve this by producing compact shares of size $\mathcal{O}(\log N)$ or $\mathcal{O}(\lambda \log N)$, where λ is a security parameter. For example, if $N = 1$ billion, a 1-hot vector would take about 1 GB, but a DPF share may only need 30–100 bytes. This emphasizes the importance of transitioning to DPFs for enhanced security and efficiency.

Definition 2.2. A $(2, 2)$ -distributed point function, or $(2, 2)$ -DPF, consists of two probabilistic polynomial-time (PPT) algorithms, **Generate** and **Evaluate**, which define secret-shared representations of point functions with domain $j \in [0, m)$. Given the following inputs:

1. A security parameter $\kappa \in \mathbb{N}$,
2. A target index j^* ,
3. A target value T ,

the system behaves as follows:

1. **Correctness:** When the key pair $(p_0, p_1) \leftarrow \text{Generate}(1^\kappa, j^*, T)$ is generated, for all $j \in [0, m)$, the following holds:

$$\text{Evaluate}(p_0, j) \oplus \text{Evaluate}(p_1, j) = \begin{cases} T & \text{if } j = j^*, \\ 0 & \text{otherwise.} \end{cases}$$

2. **Simulatability:** There exists a PPT simulator $\mathcal{R}$ such that, for any tuple of target index and target value (j^*, T) , and a bit $b \in \{0, 1\}$, the distributions of $\{\mathcal{R}(1^\kappa, b)\}_{\kappa \in \mathbb{N}}$ and $\{p_b | (p_0, p_1) \leftarrow \text{Generate}(1^\kappa, j^*, T)\}_{\kappa \in \mathbb{N}}$ are computationally indistinguishable.

The keys p_b output by **Generate** are called $(2, 2)$ -DPF keys.

In the context of a DPF, the point function $f_{\gamma, \delta}$ is split into two secret shares, share_A and share_B , which are distributed among two parties. These shares individually reveal no information about the secret index γ or the non-zero value δ . To evaluate the function at any input y , each party independently applies the cryptographic

evaluation algorithm **Evaluate** to their respective share, and their results are summed to reconstruct the output:

$$\text{Evaluate}(\text{share}_A, y) + \text{Evaluate}(\text{share}_B, y) = f_{\gamma, \delta}(y).$$

This guarantees correctness: the result equals δ when $y = \gamma$, and 0 otherwise, while keeping both γ and δ secret.

2.3 DPF Construction

As we have seen, point functions can be represented as a binary tree, and DPFs utilize this idea to share the function efficiently across two parties. A Distributed Point Function (DPF) is defined by two probabilistic polynomial-time (PPT) algorithms: **Gen** (generation) and **Eval** (evaluation).

In the **Gen** algorithm, the input consists of the target index i^* , the corresponding output value M , and a security parameter λ . The algorithm starts by generating two random seeds—one for each party—which serve as the labels for the roots of two binary trees of height $h = \log n$, where n is the domain size. Each node in the tree contains a λ -bit label and a single-bit flag called the advice bit.

As the tree is built level by level, a cryptographic Pseudorandom Generator (PRG) is used to expand a node's label into the labels of its two children (left and right). This expansion is deterministic, ensuring the same input label always leads to the same child labels.

To ensure that only one path in the tree is different between the two parties (the path that leads to the target index i^*), the construction uses special values called **correction words**.

At each level of the tree (except the leaves), there is exactly one node that must be different between the two parties. This is done by applying a correction word to that node. The correction word changes the node's label and its advice bit so that the tree paths start to differ only at the right place.

If a node's advice bit is set to 1, then the correction word for that level is XORed with the PRG output, modifying the label and advice bits of the child nodes. This

controlled modification ensures that at every level, there is exactly one diverging path, while all other nodes remain identical across both trees.

Finally, at the leaf level, a last correction word i.e is Final Correction word is applied so that the two parties' outputs differ by the target value M only at index i^* , enabling correct evaluation of the point function while preserving privacy.

The output of the **Gen** algorithm consists of two DPF keys, denoted as k_0 and k_1 , one for each party. Each key contains the following components:

- A random seed used to generate the root label and advice bit.
- A sequence of $h = \log n$ correction words, one for each non-leaf level of the tree. These are shared between both keys.
- A final correction word applied at the leaf level to ensure the difference in outputs at index i^* equals the target value M .

$$k_b = (s_b, \{cw_j\}_{j=1}^h, cw_{\text{final}}), \quad \text{for } b \in \{0, 1\}$$

The keys are designed such that, when each participant independently executes the **Evaluate** algorithm using their respective key and an input j , the results from both participants combine to produce the value of the point function at j , i.e.,

$$\text{Evaluate}(p_0, j) + \text{Evaluate}(p_1, j) = f_{j^*, T}(j).$$

Specifically, the sum equals T when $j = j^*$, and 0 otherwise. This guarantees correctness while preserving privacy, as each individual key reveals no information about the target index j^* or the value T .

Figure 2.2 illustrates the construction of the distributed point function (DPF). To evaluate a DPF over its entire domain, the **EVALFULL** procedure can be employed efficiently. This procedure performs a depth-first traversal of the DPF tree, which significantly reduces the average cost of pseudorandom generator (PRG) evaluations from $O(\log m)$ to $O(1)$ for each leaf node [6]. The output of the evaluation is a pair $(v_b, t_b) \leftarrow \text{EVALFULL}(p_b)$, where v_b is a vector containing the labels of all the leaf nodes in tree b , and t_b is the corresponding vector of flags. These outputs satisfy the conditions $t_0 \oplus t_1 = e_{j^*}$ and $v_0 \oplus v_1 = T \cdot e_{j^*}$, where e_{j^*} is a unit vector with a 1

at position j^* . Additionally, for any index j , the leaf value $v_b[j]$ corresponds to the result of the single-point evaluation $\text{EVAL}(p_b, j)$.

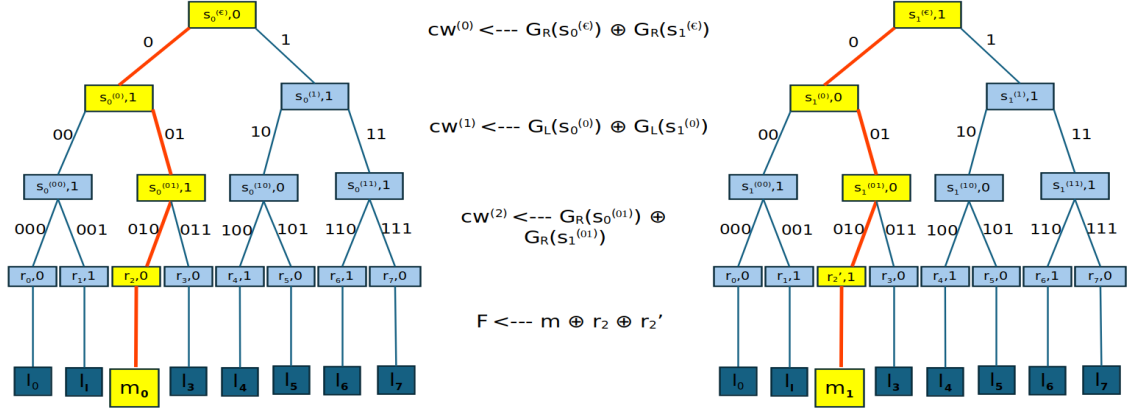


FIG. 2.2: The two trees illustrate the DPF structures maintained by P_0 and P_1 . Each node contains a tuple $(s^{(x)}, b)$, where $s^{(x)}$ is the label and b denotes the advice bit. Child nodes are computed using a length-doubling PRG along with correction terms. Leaf nodes lie in the domain $\{0, 1\}^{\lambda \times w}$, and all corresponding nodes are identical across the two trees except along the designated path to the target leaf.

2.3.1 Incremental DPFs

An Incremental Distributed Point Function (IDPF) extends a standard DPF by allowing evaluation at prefixes of the target index i^* . In an IDPF, the two parties will get different outputs if the evaluated bitstring i is a prefix of i^* , and the same output otherwise [?].

To achieve this, a final correction word is applied at the evaluated level, ensuring correctness. the evaluation continues to the next layer by xoring correction word to that layer. While constructing an IDPF is about 50% more expensive than a regular DPF, it effectively provides $\log n$ DPFs, each corresponding to a prefix of i^* .

2.4 Secure Multi-party Computation

Secure Multiparty Computation (MPC) allows a set of parties, $Q_0, Q_1, \dots, Q_m$, each with a private input $y_0, y_1, \dots, y_m$, to collaboratively compute a function $z =$

$f(y_0, \dots, y_m)$ without revealing their individual inputs to one another. The central principle is that each participant should only learn the final result z and no additional information about the other participants' inputs.

A more basic form of MPC, called 2-party MPC (2-MPC), involves just two parties, Q_0 and Q_1 , each holding a private input. They aim to compute $f(y_0, y_1)$ while maintaining two key properties: privacy (ensuring that no extra information is leaked other than the result) and correctness (ensuring that both participants receive the correct output). Figure 2.3 shows an illustration of the 2-party MPC setup. A further extension of this approach is the (2+1)-MPC, also known as server-assisted 2-party computation, where a third party is introduced. This third party does not hold any private input or receive any messages, and its sole function is to assist the two primary participants by providing correlated randomness. The third party is assumed to be non-colluding and only serves to optimize the protocol's efficiency.

The main difference between a (2 + 1)-MPC protocol and a 3-party protocol is that, in the former, the third party (referred to as the "server") aids in the computation by supplying correlated randomness but does not participate in the computation itself or learn any information about the inputs or the output. Its role is strictly to enhance the protocol's efficiency. In contrast, a 3-party MPC protocol involves three fully participating parties, each with an input, and all parties actively take part in the computation and communication during the protocol.

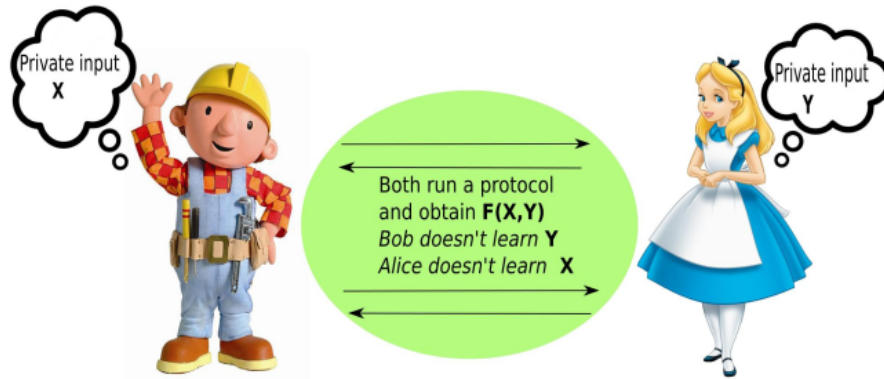


FIG. 2.3: Secure Multi-Party Computation

Performance Metrics of an MPC Protocol

When assessing the efficiency of a Secure Multiparty Computation (MPC) protocol, three core metrics are typically considered. The first is the *computation overhead*, referring to the amount of local processing each party performs, such as encrypting inputs or evaluating circuits. The second is the *communication overhead*, which captures the total volume of data exchanged between the parties. The third is the *round complexity*, measuring the number of interactive steps (i.e., rounds of message exchanges) required to execute the protocol. These criteria are critical for evaluating the practical deployment and scalability of MPC schemes.

The (2+1)-party MPC protocols implemented in this work are detailed in Sections 2.5, 2.6, and 2.7. In these protocols, an auxiliary party (the “+1”) assists by supplying **correlated randomness**, in the form of either *AND tuples* or *dot-product tuples*. These are distributed as (U_0, V_0, W_0) to party A_0 and (U_1, V_1, W_1) to party A_1 , with the specific structure depending on the tuple type.

The following definitions for the tuples are adapted from Duoram [7]:

(AND Tuples): The AND tuples (U_0, V_0, W_0) and (U_1, V_1, W_1) are each comprised of λ -bit strings. For a uniformly random bit string δ , the following conditions are satisfied:

1. $W_0 = (U_0 \wedge V_1) \oplus \delta$
2. $W_1 = (U_1 \wedge V_0) \oplus \delta$

(Dot-Product Tuples): The dot-product tuples consist of $(\vec{M}_0, \vec{N}_0, R_0) \in \mathbb{Z}_{2^w}^n \times \mathbb{Z}_{2^w}^n \times \mathbb{Z}_{2^w}$ and $(\vec{M}_1, \vec{N}_1, R_1) \in \mathbb{Z}_{2^w}^n \times \mathbb{Z}_{2^w}^n \times \mathbb{Z}_{2^w}$, satisfying the following for a random $\tau \in \mathbb{Z}_{2^w}$:

- (i) $R_0 = \langle \vec{M}_0, \vec{N}_1 \rangle + \tau$
- (ii) $R_1 = \langle \vec{M}_1, \vec{N}_0 \rangle - \tau$

All values are sampled uniformly at random from their respective domains, ensuring both security and correctness of computations over Boolean or arithmetic operations.

2.5 MPC Protocol for Bitwise AND

1. Suppose parties A_0 and A_1 hold private inputs (a_0, b_0) and (a_1, b_1) , respectively. They aim to jointly compute additive shares of $(a_0 \oplus a_1) \wedge (b_0 \oplus b_1)$. Precomputed AND tuples (U_0, V_0, W_0) and (U_1, V_1, W_1) are used.
2. Each party $A_i \in \{0, 1\}$ computes the masked inputs $a_i \oplus U_i$ and $b_i \oplus V_i$, and sends them to the other party.
3. Upon receiving the masked inputs, both parties locally compute their output shares:

- Party A_0 :

$$r_0 = (a_0 \wedge (b_0 \oplus b_1 \oplus V_1)) \oplus (V_0 \wedge (a_1 \oplus U_1)) \oplus W_0$$

- Party A_1 :

$$r_1 = (a_1 \wedge (b_1 \oplus b_0 \oplus V_0)) \oplus (V_1 \wedge (a_0 \oplus U_0)) \oplus W_1$$

The final result satisfies:

$$r_0 \oplus r_1 = (a_0 \oplus a_1) \wedge (b_0 \oplus b_1)$$

This protocol completes in a single communication round, requiring the exchange of only one word per party.

2.6 MPC Protocol for Dot Product

The dot-product protocol follows Duoram [7].

1. Let parties A_0 and A_1 possess vector shares $(\vec{u}_0, \vec{v}_0)$ and $(\vec{u}_1, \vec{v}_1)$, respectively, such that $\vec{u} = \vec{u}_0 + \vec{u}_1$ and $\vec{v} = \vec{v}_0 + \vec{v}_1$. Their objective is to compute additive shares of $\langle \vec{u}, \vec{v} \rangle$. Pre-shared dot-product tuples $(\vec{M}_i, \vec{N}_i, R_i)$ are used.
2. Each party $A_i \in \{0, 1\}$ sends the masked vectors $\vec{u}_i + \vec{M}_i$ and $\vec{v}_i + \vec{N}_i$ to the other party.

3. Upon receiving these masked vectors, both parties compute:

$$\begin{aligned} s_0 &= \langle \vec{u}_0, \vec{v}_0 + \vec{v}_1 + \vec{N}_1 \rangle - \langle \vec{N}_0, \vec{u}_1 + \vec{M}_1 \rangle + R_0 \\ s_1 &= \langle \vec{u}_1, \vec{v}_1 + \vec{v}_0 + \vec{N}_0 \rangle - \langle \vec{N}_1, \vec{u}_0 + \vec{M}_0 \rangle + R_1 \end{aligned}$$

The reconstructed output satisfies:

$$s_0 + s_1 = \langle \vec{u}, \vec{v} \rangle$$

This protocol achieves its goal with a single round of communication, involving only one word of data per party.

2.7 MPC Protocol for Arithmetic Multiplication

The multiplication protocol, adapted from Duoram [7], is outlined below:

1. Assume parties A_0 and A_1 hold values (m_0, n_0) and (m_1, n_1) , respectively. The goal is to compute shares t_0, t_1 such that:

$$t_0 + t_1 = (m_0 + m_1)(n_0 + n_1) \mod p$$

where p is a public prime. Precomputed Beaver triples (r_0, s_0, q_0) , (r_1, s_1, q_1) are used, satisfying:

$$r_0 + r_1 = r, \quad s_0 + s_1 = s, \quad q_0 + q_1 = rs \mod p$$

2. Each party computes masked differences:

$$\delta_i = m_i - r_i \mod p, \quad \epsilon_i = n_i - s_i \mod p$$

and exchanges these values.

3. After the exchange, both parties compute:

$$\delta = \delta_0 + \delta_1 \mod p, \quad \epsilon = \epsilon_0 + \epsilon_1 \mod p$$

4. Finally, the output shares are computed as:

- Party A_0 : $t_0 = q_0 + \delta s_0 + \epsilon r_0 + \delta \epsilon \mod p$
- Party A_1 : $t_1 = q_1 + \delta s_1 + \epsilon r_1 \mod p$

The correctness condition:

$$t_0 + t_1 = (m_0 + m_1)(n_0 + n_1) \mod p$$

This multiplication protocol also concludes in one round, with each party exchanging two field elements ($2w$ bits total).

2.8 Distributed Oblivious RAMs (DORAMs)

So far, we have explored various MPC protocols that handle basic operations such as multiplication, dot product, and bitwise AND. However, we have not yet covered how reads and writes are performed in the MPC setting. This section addresses the concept of *oblivious reads* and *oblivious writes*, which are crucial for establishing secure data access methods in privacy-preserving systems.

Oblivious RAM (ORAM) is a cryptographic method designed to conceal access patterns to data, ensuring that the underlying storage infrastructure cannot reveal which specific data items are being accessed. However, traditional ORAM has inherent limitations related to scalability and efficiency, especially in single-server setups.

Distributed Oblivious RAM (DORAM) is an extension of ORAM that operates in a *multi-server environment*, allowing for secure data access across distributed servers. In this system, no individual server learns either the **accessed data** or the **access patterns**, such as the specific memory locations being read or written. By distributing the trust across multiple servers, DORAM not only strengthens security (by mitigating the risks associated with compromised servers) but also improves scalability (by reducing the communication overhead compared to single-server ORAM).

Within DORAM, the data is divided into *secret shares*, which are distributed across several servers. When a user needs to read or modify data, the user sends secret shares of the query to the servers. Each server only processes its part of the query

without having access to the full information about the data being requested. The servers then return their results, which the user combines to obtain the final output or update the data. This approach makes DORAM particularly effective in secure multiparty computation (MPC) applications.

2.8.1 Floram and Duoram

Distributed Oblivious RAM (DORAM) has seen significant advancements with two notable developments: FLORAM (CCD 2017) and DUORAM (USENIX Security 2023). FLORAM was one of the first DORAM protocols to use Distributed Point Functions (DPFs) to compactly represent read and write operations. It employed a hybrid memory model that combined encrypted memory for reading and XOR-shared memory for writing, which helped reduce computational overhead. However, FLORAM’s communication cost scales as $O(mn)$ for m operations on a memory of size n , making it inefficient for large datasets. Additionally, it required periodic memory refreshes which cost $O(n)$ in computation and communication. However, they could amortize this cost to have a $O(\sqrt{n})$ computational and communication cost stemming from the refresh operation. Building on FLORAM, DUORAM introduced several key innovations that made it far more efficient and scalable. DUORAM reduces communication costs to $O(m \log n)$ by using a novel dot-product evaluation technique for secret-shared vectors. It also eliminates the need for periodic memory refreshes, significantly improving performance for large-scale applications. Most of DUORAM’s heavy computation is shifted to a preprocessing phase, which minimizes communication during actual read/write operations. Furthermore, DUORAM supports both two-party protocols (using Symmetric PIR) and three-party protocols (with single-round interaction). In this work, we build upon DUORAM as the foundation for our oblivious trie data structure.

Chapter 3

PRAC

Private Random Access Computation (PRAC) is a three-party framework designed for secure multiparty computation (MPC). The three parties involved are Q_0 , Q_1 , and Q_2 . Among them, Q_0 and Q_1 hold secret-shared data, while Q_2 does not possess any actual data but plays a crucial role in providing randomness to the other parties.

PRAC performs intensive local computation and incurs high bandwidth usage in *Distributed Oblivious RAM (DORAM)* to enable secure memory access and hence leverages *Duoram* due to its efficient setup. Duoram operates in two distinct phases: a *preprocessing phase* and an *online phase*. During preprocessing, several heavy computations are carried out, such as generating DPFs at random indices (as described by Doerner and Shelat [8]), generating *AND triples* and *dot product triples* [9, 10], producing correlated randomness, and more. The online phase, in contrast, performs lightweight computations, uses constant bandwidth and limited rounds, relying on operations such as *circular shifts* [7] to access indices.

In Duoram, each party initializes its local memory to zero, thereby eliminating initialization costs. The helper party Q_2 not only supplies correlated randomness but also actively participates in memory read and update operations during the online phase. By structuring the computation as *coroutines*, PRAC ensures that subcomputations run independently and messages are batched together, which helps reduce latency and improve overall efficiency by minimizing communication

overhead. Furthermore, PRAC extends Duoram by supporting different memory-sharing schemes, including *XOR-sharing* and *additive-sharing*, making it adaptable to a broad range of applications.

The security model assumes a *semi-honest adversary* and relies on three core assumptions: (i) the existence of *pseudorandom generators (PRGs)* for DPFs, (ii) *secure communication channels* among all parties, and (iii) *no collusion* among Q_0 , Q_1 , and Q_2 . These guarantees enable PRAC to maintain a balance between security, efficiency, and low-latency secure computation.

In this framework, the database is represented as $B \in \{0, 1\}^{m \times z}$, where m denotes the number of entries and z represents the word size. The database is secret-shared such that $B_0 + B_1 = B$, where B_0 and B_1 are additive shares modulo 2^z . A database entry at index j is represented as $B[j]$, with its corresponding shares held by Q_0 and Q_1 .

PRAC utilizes *blinded shares*, where the blinding factors are held by Q_2 . These blinded shares are computed as $B'_b = B_b + \zeta_b$ for $b = 0, 1$ and $\zeta \in \{0, 1\}^{m \times z}$, ensuring secure memory access without revealing sensitive information. During read operations, the primary parties exchange these blinded shares to securely retrieve the desired data.

$$B_0 = 0, \quad \zeta_0 = 0; \quad B_1 = 0, \quad \zeta_1 = 0.$$

PRAC enhances secure multiparty computation by reducing communication overhead, lowering latency, and ensuring secure access to memory. Its modular design and flexibility make it well-suited for a variety of privacy-preserving applications.

3.1 DORAM operations in PRAC

In PRAC's DORAM protocol, three parties (Q_0 , Q_1 , Q_2) cooperate through two phases. During the preprocessing phase, Q_2 generates Distributed Point Functions (DPFs) for a random index and distributes correlated randomness (such as Beaver triples) along with a blinded share of the database to Q_0 and Q_1 . In the online phase, the parties perform a cyclic shift on the preprocessed DPFs to align them

with the target index i^* , which results in secret-shared word vectors (encoding the update value N) and flag vectors (tracking access patterns). The blinded share and preprocessed data provided by Q2 ensure obliviousness (by concealing i^*). Figure 3.1 illustrates the DORAM setup.

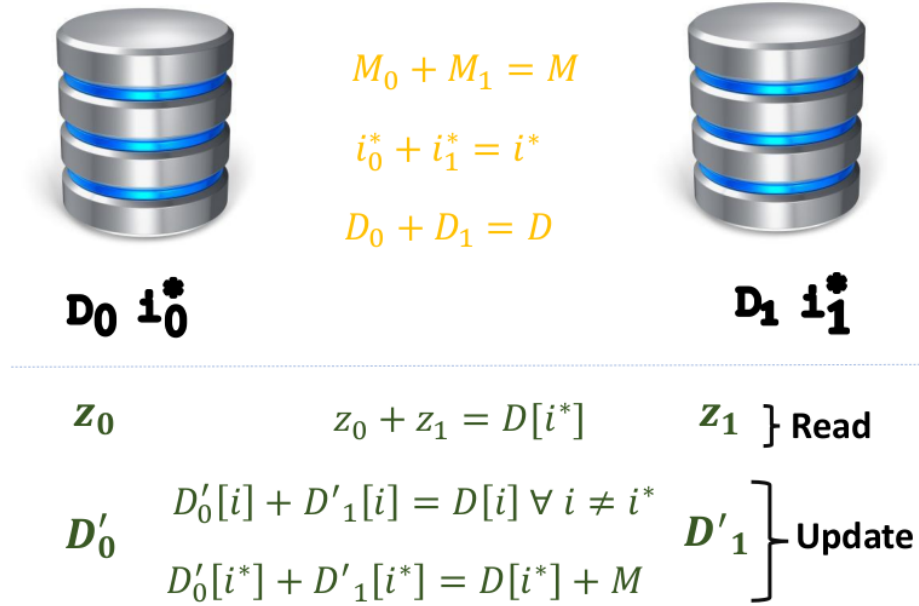


FIG. 3.1: DORAM Read and Update operations

3.1.1 3P-DUORAM Read Protocol

In the 3-party DUORAM read protocol, party Q0 holds $(C_0, \omega_0, \bar{C}_1, i_0^*)$, party Q1 holds $(C_1, \omega_1, \bar{C}_0, i_1^*)$, and party Q2 holds (ω_0, ω_1) . The output of Q0 and Q1 is to obtain shares fetch_0 and fetch_1 , such that

$$\text{fetch}_0 + \text{fetch}_1 = C[i^*],$$

where C is the secret-shared database and $i^* = i_0^* + i_1^*$.

Preprocessing Phase: Initially, Q0 and Q1 select random index shares s_{i0} and s_{i1} , respectively. With the help of Q2, they collaboratively execute a (2+1)-party MPC protocol [8] to generate three Distributed Point Functions (DPFs) for the index $s_i = s_{i0} + s_{i1}$. Each party receives DPF keys: Q0 holds $(\bar{k}_0^{(1)}, \bar{k}_0^{(2)}, \bar{k}_0^{(3)})$, and Q1 holds $(\bar{k}_1^{(1)}, \bar{k}_1^{(2)}, \bar{k}_1^{(3)})$. Then, Q0 sends $\bar{k}_0^{(2)}$ to Q2, and Q1 sends $\bar{k}_1^{(3)}$ to Q2. The parties evaluate the DPFs and obtain XOR-shared flag vectors: Q0 and Q1 receive $\tilde{f}_0^{(k)}$ and

$\tilde{f}_1^{(k)}$ for $k \in \{1, 2, 3\}$, while Q2 receives $\tilde{f}_0^{(2)}$ and $\tilde{f}_1^{(3)}$. These XOR shares are then locally converted to additive shares by all parties as it is easier to perform a dot product and also optimizes bandwidth.

Online Phase: Parties Q0 and Q1 exchange their offset values $i_0^* - s_{i0}$ and $i_1^* - s_{i1}$, and also send these values to Q2. Together, they reconstruct the total shift value

$$T = (i_0^* - s_{i0}) + (i_1^* - s_{i1}).$$

Then, the parties shift their additive shares: Q0 computes $f_0^{(k)}[i] \leftarrow f_0^{(k)}[i - T]$ for $k \in \{1, 3\}$; Q1 performs the same for $k \in \{1, 2\}$; and Q2 shifts $f_0^{(2)}$ and $f_1^{(3)}$. Next, Q2 samples a random mask $\rho \in \{0, 1\}^w$ and sends masked values:

$$\gamma_0 = -\langle \omega_0, f_1^{(3)} \rangle + \rho$$

to Q0, and

$$\gamma_1 = -\langle \omega_1, f_0^{(2)} \rangle - \rho$$

to Q1.

Finally, Q0 computes

$$\text{fetch}_0 = \langle C_0 + \bar{C}_1, f_0^{(1)} \rangle - \langle \omega_0, f_0^{(3)} - f_0^{(1)} \rangle + \gamma_0,$$

and Q1 computes

$$\text{fetch}_1 = \langle C_1 + \bar{C}_0, f_1^{(1)} \rangle - \langle \omega_1, f_1^{(2)} - f_1^{(1)} \rangle + \gamma_1,$$

thus securely reconstructing the desired database value in a secret-shared manner. This read protocol is restated from [7].

3.1.2 3P-DUORAM Update Protocol

In the 3P-DUORAM UPDATE protocol, the system operates with three parties: Q0, Q1, and Q2. Initially, Q0 holds $(C_0, \omega_0, \bar{C}_1, i_0^*, N_0)$, Q1 holds $(C_1, \omega_1, \bar{C}_0, i_1^*, N_1)$, and Q2 holds (ω_0, ω_1) . At the end of the protocol, Q0 receives $(C'_0, \omega'_0, \bar{C}'_1)$, Q1 receives $(C'_1, \omega'_1, \bar{C}'_0)$, and Q2 obtains (ω'_0, ω'_1) . These outputs satisfy the correctness

conditions:

$$\bar{C}'_0 = C'_0 + \omega'_0, \quad \bar{C}'_1 = C'_1 + \omega'_1, \quad C'_0 + C'_1 = C_0 + C_1 + (N_0 + N_1) \cdot e_{i_0^* + i_1^*}.$$

Preprocessing Phase: The preprocessing phase begins with parties Q0 and Q1 independently choosing random index shares t_{i0} and t_{i1} , respectively. Together with the assistance of Q2, they run a (2+1)-party MPC protocol to generate three Distributed Point Functions (DPFs): $(\bar{k}_0^{(1)}, \bar{k}_0^{(2)}, \bar{k}_0^{(3)})$ and $(\bar{k}_1^{(1)}, \bar{k}_1^{(2)}, \bar{k}_1^{(3)})$, with index $t_i = t_{i0} + t_{i1}$. During this process, Q0 and Q1 also obtain respective shares $G_0^{(k)}$ and $G_1^{(k)}$ of the final correction words for $k \in \{1, 2, 3\}$. Next, Q0 sends $\bar{k}_0^{(2)}$ to Q2, while Q1 sends $\bar{k}_1^{(3)}$ to Q2.

Subsequently, all parties evaluate the DPFs to obtain XOR-shared flag bits (flag vectors) and leaves (word vectors). For each $k \in \{1, 2, 3\}$, Q0 and Q1 obtain $\tilde{f}_0^{(k)}, \tilde{v}_0^{(k)}$ and $\tilde{f}_1^{(k)}, \tilde{v}_1^{(k)}$, respectively. Q2 receives $(\tilde{f}_0^{(2)}, \tilde{v}_0^{(2)})$ and $(\tilde{f}_1^{(3)}, \tilde{v}_1^{(3)})$. Finally, these XOR-shared flags and leaves are converted into additive shares f and v .

The final correction words are defined as:

$$G_0^{(k)} = - \sum_i v_0^{(k)}[i] \quad \text{and} \quad G_1^{(k)} = - \sum_i v_1^{(k)}[i], \quad k \in \{1, 2, 3\}.$$

Therefore, the following condition holds:

$$v_0^{(k)}[t_i] + v_1^{(k)}[t_i] + G_0^{(k)} + G_1^{(k)} = 0, \quad k \in \{1, 2, 3\}$$

3.2 Oblivious Data Structures in PRAC

The **PRAC framework** supports three privacy-preserving data structures: **binary search trees**, **heaps**, and **AVL trees**. Binary search trees enable efficient private searches with $O(\log n)$ bandwidth, while heaps are optimized for priority-based operations like insertions ($O(\log \log n)$ rounds) and extract-min ($O(\log n)$ bandwidth). AVL trees, being self-balancing, support secure insertions and deletions with $O(\log n)$ rounds. PRAC uses techniques like **Distributed Oblivious RAM (DORAM)** and **Distributed Point Functions (DPFs)** to ensure data privacy and oblivious access patterns, making it ideal for secure multi-party computation in dynamic workloads such as private databases or task scheduling.

Chapter 4

Oblivious Tries

In this chapter, we will describe the cryptographic protocols needed to implement Oblivious Tries. The chapter will first outline the memory layout of the Tries in Section 4.1, and then we will cover the implementation process.

4.1 Memory Layout

Let Σ denote a finite set of symbols, referred to as the *alphabet*, consisting of exactly α elements. We write:

$$\Sigma = \{a_1, a_2, \dots, a_\alpha\}.$$

Definition 4.1 (Word). A *word* over an alphabet Σ is defined as a finite ordered sequence composed of symbols from Σ . The collection of all such words is represented by Σ^* . That is,

$$w \in \Sigma^* \iff w = a_1 a_2 \dots a_k, \quad \text{where each } a_j \in \Sigma.$$

The *length* of a word w , written as $|w|$, indicates the total number of symbols it contains. Specifically,

$$\text{If } w = a_1 a_2 \dots a_n, \text{ then } |w| = n.$$

Definition 4.2 (Integer Encoding of Symbols). Let $\phi : \Sigma \rightarrow \mathbb{Z}$ be an *injective function* that maps each symbol from the alphabet Σ to a distinct integer. That is,

$$\forall a, b \in \Sigma, a \neq b \Rightarrow \phi(a) \neq \phi(b).$$

Given a word $w = a_1 a_2 \dots a_n \in \Sigma^*$, the function ϕ can be extended to encode the entire word as a sequence of integers:

$$\phi(w) = (\phi(a_1), \phi(a_2), \dots, \phi(a_n)) \in \mathbb{Z}^n.$$

Definition 4.3 (Dictionary). A *dictionary* over an alphabet Σ is defined as a finite collection of words formed using the symbols from Σ . Formally,

$$\mathcal{D} \subseteq \Sigma^*$$

Each element $w \in \mathcal{D}$ represents a valid word contained within the dictionary.

4.1.1 Tree-Based Representation of a Dictionary

We organize the dictionary as an α -ary tree, where each node—referred to as a *block*—stores a bit vector of length α . The tree structure mirrors the character positions in the words it encodes.

- The **root level** aligns with the first symbol in any word.
- The k^{th} level corresponds to the k -th character in a word.

Each block is named based on its unique position in the tree, determined by the path from the root:

The root block is labeled b^ϵ .

- The α blocks at depth 1 (i.e., the first level of children) are $b^1, \dots, b^\alpha$.
- The children of the j^{th} block at depth 1 are labeled as $b^{j\|1}, \dots, b^{j\|\alpha}$.
- More generally, for any sequence s of length ℓ over $[\alpha]$, the block b^s is positioned at level ℓ and branches into α children:

$$b^{s\|1}, \dots, b^{s\|\alpha}$$

Let $\Sigma = \{a_1, a_2, \dots, a_\alpha\}$ represent an alphabet of size α . Any word w of length ℓ over this alphabet can be encoded as:

$$w = k_1 k_2 \cdots k_\ell \in [\alpha]^\ell$$

where each k_i denotes the index of the i^{th} character in the alphabet, i.e., the i^{th} letter in the word is a_{k_i} .

Encoding Word Presence and Termination. Each block consists of α entries, one for each possible symbol, and each entry contains two bits:

1. **present** bit: indicates that the corresponding symbol appears as a valid next letter from this node.
2. **terminates** bit: indicates that a word terminates at this symbol when reached via this path.

The presence of a word $w = k_1 k_2 \cdots k_\ell \in [\alpha]^\ell$ in the dictionary is encoded as follows:

- In block b^ϵ , set the k_1^{th} **present** bit to 1.
- In block b^{k_1} , set the k_2^{th} **present** bit to 1.
- More generally, for any prefix $s = k_1 \| k_2 \| \cdots \| k_{i-1} \in [\alpha]^{i-1}$, set the k_i^{th} **present** bit in block b^s to 1.
- Additionally, set the k_ℓ^{th} **terminates** bit in block $b^{k_1 \| \cdots \| k_{\ell-1}}$ to 1 to indicate the word ends there.

Example. Let $\Sigma = \{A, B, C, D\}$, so $\alpha = 4$, and fix the injective mapping ϕ such that:

$$\phi(A) = 1, \quad \phi(B) = 2, \quad \phi(C) = 3, \quad \phi(D) = 4$$

Consider the word $w = \text{ADD}$, which corresponds to the sequence:

$$k_1 = \phi(A) = 1, \quad k_2 = \phi(D) = 4, \quad k_3 = \phi(D) = 4$$

The bits set in the tree are:

- In block b^ϵ , set `present`[1] = 1
- In block b^1 , set `present`[4] = 1
- In block $b^{1\parallel 4}$, set `present`[4] = 1 and `terminates`[4] = 1

Necessity of Termination Bit. The `terminates` bit is essential to distinguish between a word that ends at a certain node and one that continues further. For example, consider the dictionary containing the words `ADD` and `ADDITION`:

- The final D in `ADD` corresponds to setting `terminates`[4] = 1 in block $b^{1\parallel 4}$.
- The final N in `ADDITION` corresponds to setting `terminates`[$\phi(N)$] = 1 in the corresponding block.

However, if only `ADDITION` was present (and not `ADD`), then the `terminates` bit would be set only for N, and not for the second D. This distinction is critical for correctly identifying which words exist in the dictionary.

4.1.2 Flat Bit Array Representation

We represent the hierarchical data structure as a single flat *bit array* by performing a prefix-order traversal (i.e., pre-order traversal) of the tree and concatenating the bits of each block in traversal order.

Let this flat array be denoted by:

$$\text{Trie} \in \{0, 1\}^{2 \cdot N \cdot \alpha}$$

Where N is the number of blocks in the tree. Each of the α entries of a block is represented using two bits:

1. A **presence** bit: indicates whether the corresponding child exists (i.e., whether the corresponding bit in the original block is set).
2. A **termination** bit: indicates whether there exists a word that *terminates* at that symbol.

Thus, each entry of Trie corresponds to a pair:

$$\text{Trie}[i] = (\text{present}, \text{terminates}) \in \{0, 1\} \times \{0, 1\}$$

Indexing in the Flat Array

Let $\text{rank}(s)$ denote the lexicographic rank of the prefix s in the dictionary. Then, the index of any alphabet k_i of a string in the flat representation of the trie is given by:

$$\text{index}(s, k_i) = \alpha \times \text{rank}(s) + (k_i - 1)$$

where:

- s is the prefix just prior to the alphabet k_i ,
- α is the branching factor of the trie,
- k_i denotes the 1-based index of the alphabet in the alphabet set.

In particular, the presence of a word $k_1 k_2 \cdots k_\ell \in [\alpha]^\ell$ is represented by setting the following bits in the bit array:

$$\text{index}(\epsilon, k_1), \text{index}(k_1, k_2), \dots, \text{index}(k_1 \parallel \cdots \parallel k_{\ell-1}, k_\ell)$$

Lexicographic Rank Function

Let $s = k_1 \parallel k_2 \parallel \cdots \parallel k_d \in [\alpha]^d$. We define the lexicographic rank of s as:

$$\text{rank}(s) = \sum_{i=0}^{d-1} \alpha^i + \sum_{j=1}^d (k_j - 1) \cdot \alpha^{d-j}$$

The first term counts the number of positions at depths less than d , and the second term gives the position of s among the α^d positions at depth d , sorted lexicographically. For every word w , we define an array called the Index array. The index array is another representation of the string. we get this array using Letter to Index algorithm

Algorithm 1 LetterToIndex Algorithm

```

1: Input: Word  $s = k_1k_2 \dots k_\ell \in [\alpha]^\ell$ ; Alphabet size  $\alpha$ 
2: Output: Array indexArray[0.. $\ell$ -1] representing bit indices in the trie
3: [A1] Initialize array indexArray of size  $\ell$ 
4: [A2] rank  $\leftarrow$  0
5: for  $i = 0$  to  $\ell - 1$  do
6:    $k \leftarrow k_{i+1}$ 
7:   if  $i = 0$  then
8:     indexArray[0]  $\leftarrow k - 1$ 
9:   else
10:    [A3] rank  $\leftarrow \alpha \cdot \text{rank} + (k_i)$   $\triangleright$  Calculating the rank of prefix
11:    [A4] indexArray[ $i$ ]  $\leftarrow \alpha \cdot \text{rank} + (k - 1)$ 
12:   end if
13: end for
14: return indexArray

```

Here, the algorithm begins by initialising an array `indexArray` (see Step **A1**) to store the bit indices of the prefixes. Then, the variable `rank` is set to 0 (Step **A2**) to track the lexicographic position of each prefix.

The loop iterates over each symbol of the input string s . For the first symbol, the bit index is directly calculated. For subsequent symbols, the rank is calculated for the prefix (Step **A3**) and the corresponding index is computed (Step **1.A4**). Here the rank of the prefix will help us to calculate the index. Finally, the algorithm returns the index array.

Example. Consider the word ADD, when the alphabet size is $\alpha = 4$.

- The symbol A corresponds to $k_1 = 1$, and the `index(0, 1)` is 0 .
- The first D corresponds to $k_2 = 4$. The rank of prefix 1 is 1, so its index is $\alpha \cdot 1 + (4 - 1) = 4 + 3 = 7$, hence the `index(1, 4)` is 7.
- The second D corresponds to $k_3 = 4$. The rank of prefix 1||4 is 8, so the index is $\alpha \cdot 8 + (4 - 1) = 32 + 3 = 35$. If this is a terminal node, the `index(8, 4)` is 35.

Figure 4.1 illustrates the flat array representation of a trie. In this example, we consider the alphabet $\Sigma = \{A, B, C, D\}$. By traversing the trie from the root and following all the bits set to 1, along with checking the termination bits, we can recover all the words stored in the database. The current set of words present in the

trie includes ADD, BAD, CAB, and DAD, each of which has its corresponding termination bit set.

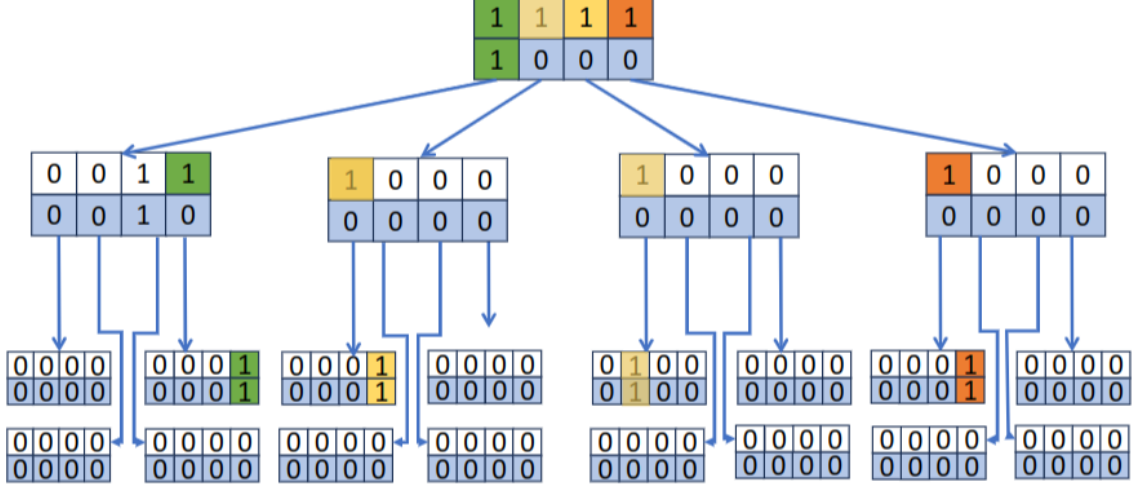


FIG. 4.1: Flat Array Representation of Trie.

4.2 Oblivious Tries

After describing the memory layout, we now describe our Distributed ORAM-based Oblivious Trie construction.

We consider two main parties, namely, P_0 and P_1 , who hold shares of the dictionary represented as a trie data structure. Each party holds a share of the flat bit array $\text{Trie} \in \{0, 1\}^{2 \cdot N \cdot \alpha}$, where each entry in Trie consists of a **presence** and **termination** bit as described in Section 4.1.2.

To insert an ℓ -length word into the dictionary, we perform ℓ oblivious writes. Note that these are *oblivious writes*, not mere updates, since the write location must be hidden from the server. Each oblivious write corresponds to a symbol in the word, and the exact index for the write is determined by the **LetterToIndex** algorithm (Algorithm 1). This algorithm outputs an array of indices, which are then split between the two parties, ensuring that each party receives a share of the index.

4.2.1 Basic Implementations

4.2.1.1 Oblivious Insert

The basic insert algorithm operates as follows:

1. For each index in the `indexArray`, the parties perform an oblivious write to set the presence bit.
2. Additionally, the `termination` bit corresponding to the last index is set to 1 to mark the end of the word.

Algorithm 2 Basic Oblivious-Insert Algorithm

- 1: **Input:** Alphabet size α , Maximum depth m , Player index $P_b \in \{0, 1\}$, Shared `indexArray` `IndexArrayXS[0... $\ell$ -1]`
 - 2: **for** $j = 0$ **to** $\ell - 1$ **do**
 // Set presence bit obliviously
 - 3: `TriebXS[IndexArrayXS[ $j$ ]] · presence` $\leftarrow P_b$
 - 4: **end for**
 // Set termination bit for last symbol
 - 5: `TriebXS[IndexArrayXS[ $\ell - 1$ ]] · termination` $\leftarrow P_b$
-

4.2.1.2 Oblivious Search

Oblivious Search works in the following setting. P_0 and P_1 hold shares of the trie datastructure. Furthermore, the get shares of a word. Their goal is to learn if the word exists in this datastructure.

Lemma 4.4. *A word exists in the dictionary if and only if*

1. *for every index that exists in the `indexArray`, the value in trie datastructure is Set.*
2. *For last index in the `indexArray`, the termination bit is also set.*

The main idea of oblivious search is as follows.

1. For every index that exists in the `IndexArray`, we read the presence bits

Algorithm 3 Basic Oblivious-Search Algorithm

```

1: Input: Alphabet size  $\alpha$ , Maximum depth  $m$ , Player index  $P_b \in \{0,1\}$ , Shared
   indexArray  $\text{IndexArray}^{XS}[0 \dots \ell-1]$ , Shared flat array  $\text{Trie}_b^{XS}$ 
2: Output: Search result  $z_b^{XS}$ 
   // Initialize result with first presence bit
3:  $z_b^{XS} \leftarrow \text{Trie}_b^{XS}[\text{IndexArray}^{XS}[0]] \cdot \text{presence}$ 
4: for  $j = 1$  to  $\ell - 1$  do
5:    $c_b^{XS} \leftarrow \text{Trie}_b^{XS}[\text{IndexArray}^{XS}[j]] \cdot \text{presence}$ 
   // perform mpc_and on each subsequent presence bit refer ??
6:    $z_b^{XS} \leftarrow z_b^{XS} \wedge c_b^{XS}$ 
7: end for
   // Include termination bit of the last symbol
8:  $t_b^{XS} \leftarrow \text{Trie}_b^{XS}[\text{IndexArray}^{XS}[\ell - 1]] \cdot \text{termination}$ 
9:  $z_b^{XS} \leftarrow z_b^{XS} \wedge t_b^{XS}$ 
10: return  $z_b^{XS}$ 

```

2. For the last index in the `IndexArray`, we read the presence bit as well as the termination bit.

3. All the aforementioned bits are multiplied together using an MPC AND.

Protocol 3 describes the Oblivious Search in detail.

4.2.1.3 Oblivious Delete

To perform a delete operation, all we need to do is set the termination bit of last index to 0. Even though the letters are set, the words are absent. Protocol 4

Algorithm 4 Basic Oblivious-Delete Algorithm

```

1: Input: Alphabet size  $\alpha$ , Maximum depth  $m$ , Player index  $P_b \in \{0,1\}$ , Shared
   indexArray  $\text{IndexArray}^{XS}[0 \dots \ell-1]$ 
   // Set termination bit for last symbol
2:  $\text{Trie}_b^{XS}[\text{IndexArray}^{XS}[\ell - 1]] \cdot \text{termination} \leftarrow 0$ 

```

4.2.2 Optimizations

After implementing the basic version of the oblivious trie, we observed that accessing an element in the second or third layer requires creating a DPF (Distributed Point Function) of size $N \cdot \alpha$, where N is the number of blocks. This significantly affects

both computation and communication costs. and Hence we introduce our optimized version of trie. Our main idea for optimization is based on the structure of the trie: the first DUORAM operation occurs at the root layer of size α , the second DUORAM operation at the first layer of size α^2 , and so on. Hence, instead of creating a DPF over the complete database, we generate DPFs layer by layer. To support this, we slightly modify our flat array representation of the trie. We represent the trie as a two-dimensional array, where each subarray corresponds to a different layer. The i -th layer contains α^i nodes, where α is the branching factor. Formally, we define the trie as:

$$\text{Trie}[i][j] \in \{0, 1\}, \{0 \leq i \leq l, 0 \leq j < \alpha^i\},$$

where l is the depth of the trie, and each element indicates whether a node is present at position j in layer i .

Since we are not accessing the elements as a single flat array but as a two-dimensional structure where each subarray corresponds to a specific alphabet symbol, the indexing method must also be adjusted. Algorithm 5 describes the indexing strategy used in the optimized version. In this approach, instead of calculating the rank of the prefix at each level, the index is computed directly based on the previously computed index. The remaining logic follows the same structure as described in Algorithm 1.

Algorithm 5 LetterToIndex2 Algorithm

```

1: Input: Word  $s = k_1k_2 \dots k_\ell \in [\alpha]^\ell$ ; Alphabet size  $\alpha$ 
2: Output: Array indexArray[0.. $\ell$ -1] representing bit indices in the trie
3: [A1] Initialize array indexArray of size  $\ell$ 
4: [A2] rank  $\leftarrow$  0
5: for  $i = 0$  to  $\ell - 1$  do
6:    $k \leftarrow k_{i+1}$ 
7:   if  $i = 0$  then
8:     indexArray[0]  $\leftarrow k - 1$ 
9:   else
10:    [A6] indexArray[ $i$ ]  $\leftarrow \alpha \cdot \text{indexArray}[i - 1] + (k - 1)$ 
11:  end if
12: end for
13: return indexArray

```

Figure 4.2 provides an overview of the optimized trie representation. In this design, each level of the trie is treated as subarray, which enables the construction of smaller Distributed Point Functions (DPFs), thereby improving efficiency in both computation and communication.

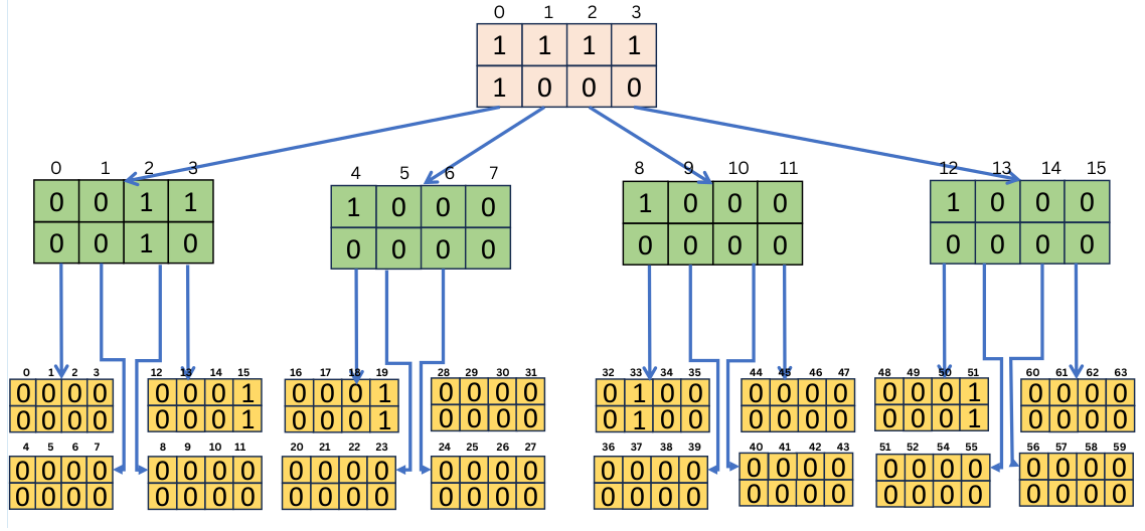


FIG. 4.2: Optimized Trie Representation

4.2.2.1 Optimized Insert

The core idea behind the optimized insert operation is as follows: we iterate through each subarray of the two-dimensional trie structure, performing a **DUORAM Write** on the presence bit at the index specified by the `indexArray`. Finally, we perform a **DUORAM Write** on the termination bit at the last index in the `indexArray`. Protocol 6 describes the optimized insert.

Algorithm 6 Optimized Oblivious-Insert Algorithm

- 1: **Input:** Alphabet size α , Maximum depth m , Player index $P_b \in \{0,1\}$, Shared `indexArray` $\text{IndexArray}^{XS}[0 \dots \ell-1]$
 - 2: **for** $j = 0$ **to** $\ell - 1$ **do**
// Set presence bit obviously
 - 3: $\text{Trie}_b^{XS}[j][\text{IndexArray}^{XS}[j]] \cdot \text{presence} \leftarrow P_b$
 - 4: **end for**
// Set termination bit for last symbol
 - 5: $\text{Trie}_b^{XS}[\ell - 1][\text{IndexArray}^{XS}[\ell - 1]] \cdot \text{termination} \leftarrow P_b$
-

4.2.2.2 Optimized Search

The idea of optimized search is also similar to the search protocol. The first letter of the word will be found in the root layer. Similarly, the i^{th} letter would be in the i^{th} layer. In other words, the first letter's presence would be present between $[1, \alpha]$. The second letter would be present between $[\alpha + 1, \alpha^2 + \alpha]$ and so on. In other words, we

need to perform, h Duoram reads on databases of size $\alpha, \alpha^2, \dots, \alpha^h$. Also, note that all the h read operations can be done in parallel. Protocol 7 describes the optimized search.

Algorithm 7 Optimized Oblivious-Search Algorithm

```

1: Input: Alphabet size  $\alpha$ , Maximum depth  $m$ , Player index  $P_b \in \{0, 1\}$ , Shared
   indexArray  $\text{IndexArray}^{XS}[0 \dots \ell-1]$ , Shared flat array  $\text{Trie}_b^{XS}$ 
2: Output: Search result  $z_b^{XS}$ 
   // Initialize result with first presence bit
3:  $z_b^{XS} \leftarrow \text{Trie}_b^{XS}[0][\text{IndexArray}^{XS}[0]] \cdot \text{presence}$ 
4: for  $j = 1$  to  $\ell - 1$  do
5:    $c_b^{XS} \leftarrow \text{Trie}_b^{XS}[j][\text{IndexArray}^{XS}[j]] \cdot \text{presence}$ 
   // perform mpc_and on each subsequent presence bit refer ??
6:    $z_b^{XS} \leftarrow z_b^{XS} \wedge c_b^{XS}$ 
7: end for
   // Include termination bit of the last symbol
8:  $t_b^{XS} \leftarrow \text{Trie}_b^{XS}[\ell - 1][\text{IndexArray}^{XS}[\ell - 1]] \cdot \text{termination}$ 
9:  $z_b^{XS} \leftarrow z_b^{XS} \wedge t_b^{XS}$ 
10: return  $z_b^{XS}$ 

```

4.2.2.3 Optimized Oblivious Delete

The optimized delete algorithm functions similarly to the basic delete algorithm. The key difference is that it sets the termination bit to 0 at the last subarray, using the final index from the `indexArray`.

Algorithm 8 Optimized Oblivious-Delete Algorithm

- 1: **Input:** Alphabet size α , Maximum depth m , Player index $P_b \in \{0, 1\}$, Shared `indexArray` $\text{IndexArray}^{XS}[0 \dots \ell-1]$
// Set termination bit for last symbol
 - 2: $\text{Trie}_b^{XS}[\ell-1][\text{IndexArray}^{XS}[\ell-1]] \cdot \text{termination} \leftarrow 0$
-

Chapter 5

Evaluation

5.1 Theoretical Analysis

This section provides an analytical evaluation of the Oblivious Trie algorithms. Table 5.1 presents a comparative analysis of the basic and optimized versions. In both cases, the majority of the work is carried out during the preprocessing phase. All algorithms are implemented using coroutines, and for basic operations, the communication complexity is $\mathcal{O}(\log n)$ —primarily due to the DPF generation process. The round complexity also depends on the height of the trie, as it is incurred during the DPF generation. In contrast, the online phase incurs only a constant communication and round cost, while the computational cost involves evaluating the entire database. In the optimized version, DPFs are generated with varying sizes corresponding to layer sizes of $\alpha, \alpha^2, \dots, n$. As a result, the worst-case complexities for communication, rounds, and computation remain $\mathcal{O}(\log n)$, but with reduced constants due to more efficient DPF usage and layer-wise optimization.

TABLE 5.1: A comparative analysis of various protocols based on their communication and computation overhead.

Protocol	Communication		Computation
	Bandwidth	Rounds	
Insert (Basic)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Search (Basic)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Delete (Basic)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Insert (Optimized)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Search (Optimized)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Delete (Optimized)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

5.2 Experimental Analysis

We implemented and evaluated the Oblivious Trie in C++, using `Boost Asio` to support asynchronous network communication. All experiments were conducted on a single machine equipped with an **Intel Core i7-4770 CPU @ 3.4 GHz (4-core, 8-thread)**. Each party was executed inside a separate Docker container, pinned to a **dedicated CPU thread**. We used `netem` to simulate a network environment with 100 Mbps bandwidth and 0 ms latency between containers. The pseudorandom generators (PRGs) in the DPFs are implemented using AES, and our DORAM setup operates on 64-bit word-sized data. In these experiments, we set the alphabet size to 4, with $\Sigma = \{A, B, C, D\}$. The results obtained contain some degree of noise and are not fully smoothed out.

Evaluation of Insert Operation

Figure 5.1 and 5.2 compare the performance of oblivious trie insertions in both the basic and optimized versions under a network setup with 0 ms latency and 100 Mbps bandwidth. The evaluation focuses solely on insert operations of different lengths while varying the database size from 4^1 to 4^{10} .

Figure 5.1 illustrates the overall communication bandwidth required for both versions across increasing database sizes. It can be observed that the optimized version consistently incurs lower communication cost compared to the basic version. This improvement is primarily due to the use of DPFs with variable sizes (as discussed in section 4.2.2), which are generated during the preprocessing phase.

Figure 5.2 shows the wall-clock execution time, revealing a clear performance gap between the basic and optimized versions as the database size grows, underscoring the improved efficiency of the optimized approach.

Figure 5.3 shows the communication cost during the online phase, showing that it remains relatively constant even as the database size grows. This is because most of the communication is done in the preprocessing phase.

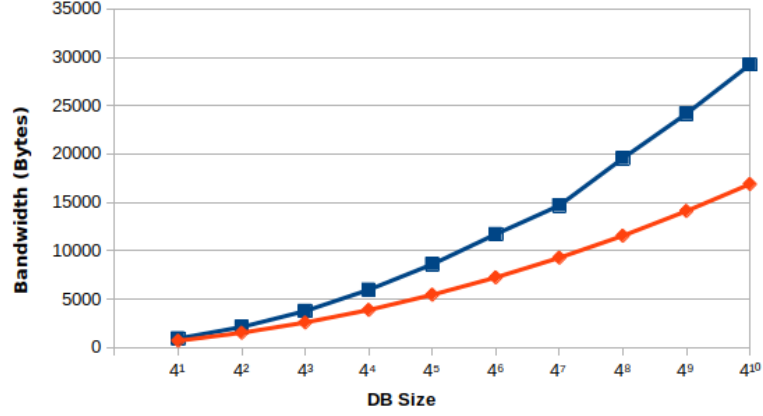


FIG. 5.1: Comparing communication cost of Oblivious Tries insert
 ■ Basic vs. ■ Optimized

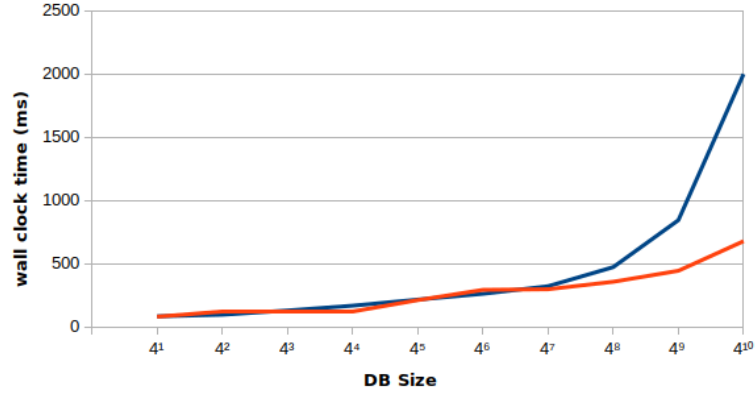


FIG. 5.2: Comparing wall clock time of Oblivious Tries insert
 ■ Basic vs. ■ Optimized

Evaluation of Search Operation

Similar to the insert operation, we also evaluated the search functionality of the oblivious trie. Figures 5.6 and 5.5 compare the basic and optimized versions in terms of wall-clock execution time and total bandwidth. As expected, the optimized version demonstrates better performance, with reduced communication overhead and faster execution, especially as the database size increases.

Figure 5.4 illustrates the online communication cost for the search operation. As expected, both the basic and optimized versions exhibit similar bandwidth usage during the online phase, since the optimizations primarily affect the preprocessing stage.

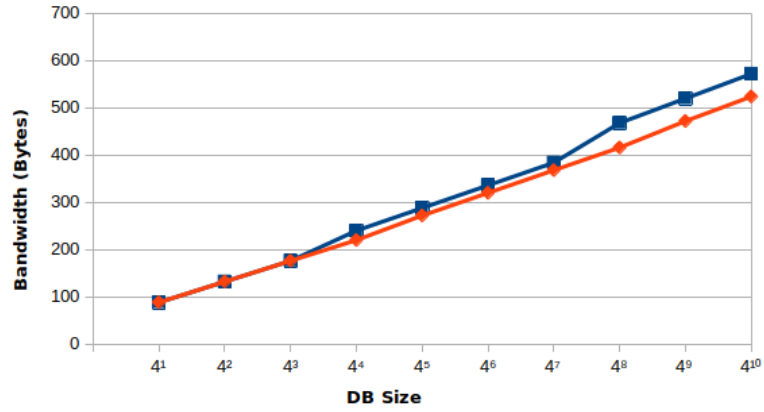


FIG. 5.3: Comparing online Phase communication cost of Oblivious Tries insert
 ■ Basic vs. ■ Optimized

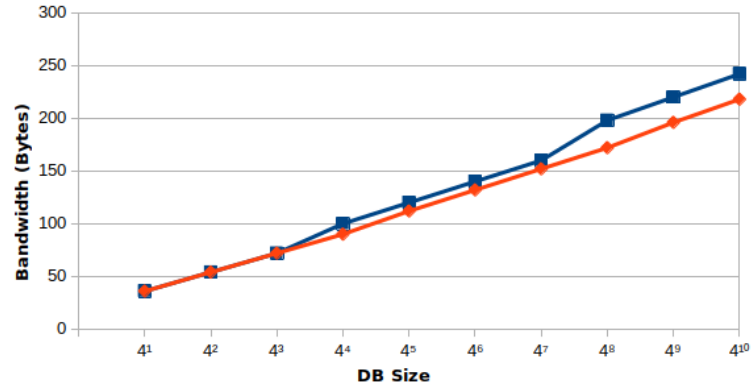


FIG. 5.4: Comparing communication cost of Oblivious Tries search
 ■ Basic vs. ■ Optimized

Evaluation of Delete Operation

Figures 5.7 and 5.8 depict the total communication bandwidth and wall-clock execution time for the delete operation. The results indicate that the optimized version performs significantly better than the basic version in terms of both communication efficiency and execution speed.

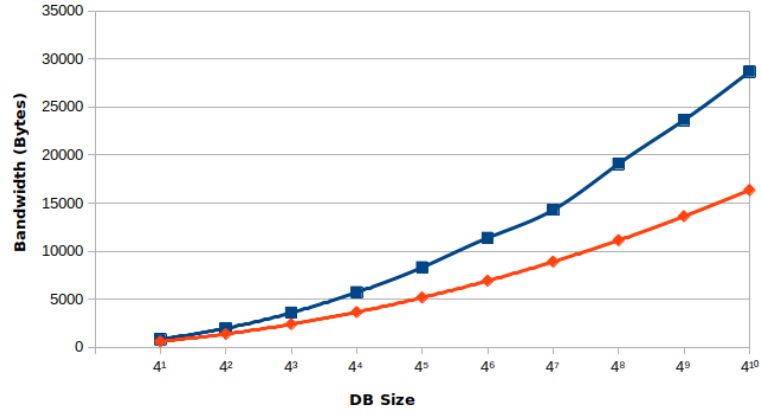


FIG. 5.5: Comparing wall clock time of Oblivious Tries search
 ■ Basic vs. ■ Optimized

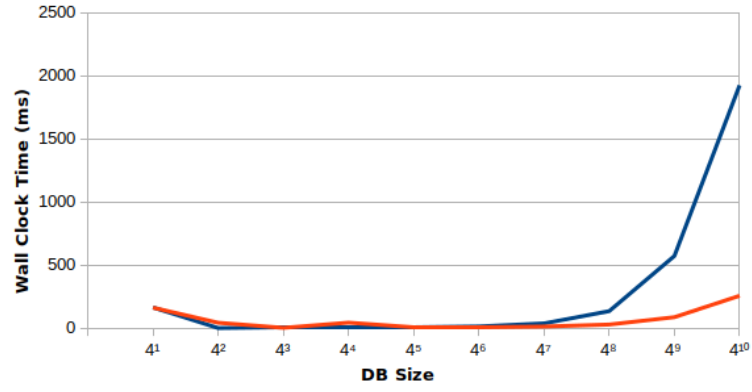


FIG. 5.6: Comparing online Phase communication cost of Oblivious Tries search
 ■ Basic vs. ■ Optimized

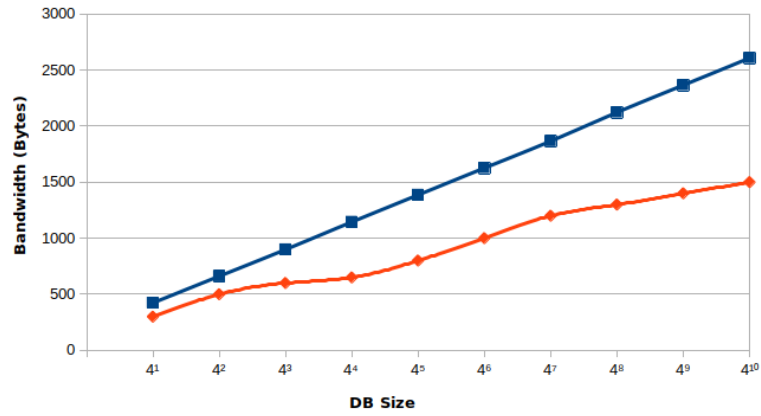


FIG. 5.7: Comparing communication cost of Oblivious Tries delete
 ■ Basic vs. ■ Optimized

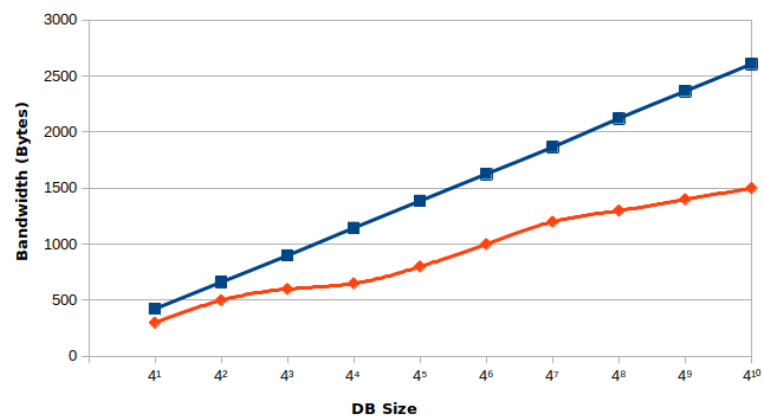


FIG. 5.8: Comparing wall clock time of Oblivious Tries delete
■ Basic vs. ■ Optimized

Chapter 6

Conclusion

This thesis presented the design, implementation, and evaluation of Oblivious Tries using both basic and optimized constructions. The core objective was to enable privacy-preserving insert and search operations while maintaining efficiency in communication and computation. By leveraging Distributed Oblivious RAM (DORAM) techniques and Distributed Point Functions (DPFs), we demonstrated how such operations can be securely performed in a multi-party computation setting.

We implemented the protocols in C++ using **Boost Asio** for asynchronous network communication. Experiments were conducted under simulated network conditions using Docker and **netem**, where each party was pinned to a dedicated CPU thread. We evaluated performance with respect to wall-clock time and communication bandwidth.

The results clearly indicate that the optimized version of the Oblivious Trie outperforms the basic version in both communication and time cost, especially as the database size increases. The improvements come from reducing the number of DPFs generated and queried by utilizing variable-length layers in the trie.

Future Work

While this work provides an efficient construction for oblivious insert and search operations, several directions remain open for future exploration:

- **Incremental DPFs:** As discussed in Section 2.3.1, the use of incremental DPFs can significantly reduce the communication overhead by requiring only a single DPF for m operations. Exploring more efficient constructions and practical applications of incremental DPFs could further optimize the performance of oblivious data structures.
- **Scalability:** We observed that the current implementation consumes significant memory, as each alphabet is stored using 8 bytes. Future work can focus on optimizing memory usage, for example by operating at the bit level, which would enable more efficient scaling to larger datasets.
- **Integration with Applications:** Applying Oblivious Tries in real-world use cases such as private autocomplete, prefix-based keyword search, and routing table lookups.

Overall, this work lays a strong foundation for efficient and secure prefix-based data access and opens up interesting avenues for further research in privacy-preserving data structures.

Bibliography

- [1] A. Beimel and Y. Ishai, “Information-theoretic private information retrieval: A unified construction,” in *Automata, Languages and Programming: 28th International Colloquium, ICALP 2001 Crete, Greece, July 8–12, 2001 Proceedings 28*, pp. 912–926, Springer, 2001.
- [2] I. Goldberg, “Improving the robustness of private information retrieval,” in *2007 IEEE Symposium on Security and Privacy (SP’07)*, pp. 131–148, IEEE, 2007.
- [3] S. M. Hafiz and R. Henry, “Querying for queries: Indexes of queries for efficient and expressive it-pir,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1361–1373, 2017.
- [4] S. M. Hafiz and R. Henry, “A bit more than a bit is more than a bit better,” *Proceedings on Privacy Enhancing Technologies*, 2019.
- [5] A. Vadapalli, K. Storrier, and R. Henry, “Sabre: Sender-anonymous messaging with fast audits,” in *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 1953–1970, IEEE, 2022.
- [6] E. Boyle, N. Gilboa, and Y. Ishai, “Function secret sharing: Improvements and extensions,” in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pp. 1292–1303, 2016.
- [7] A. Vadapalli, R. Henry, and I. Goldberg, “Duoram: A {Bandwidth-Efficient} distributed {ORAM} for 2-and 3-party computation,” in *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 3907–3924, 2023.
- [8] J. Doerner and A. Shelat, “Scaling oram for secure computation,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 523–535, 2017.

- [9] D. Beaver, “Efficient multiparty protocols using circuit randomization,” in *Advances in Cryptology—CRYPTO’91: Proceedings 11*, pp. 420–432, Springer, 1992.
- [10] W. Du and M. J. Atallah, “Protocols for secure remote database access with approximate matching,” in *E-Commerce Security and Privacy*, pp. 87–111, Springer, 2001.
- [11] D. A. Adeniyi, Z. Wei, and Y. Yongquan, “Automated web usage data mining and recommendation system using k-nearest neighbor (knn) classification method,” *Applied Computing and Informatics*, vol. 12, no. 1, pp. 90–108, 2016.
- [12] B. Balle, G. Barthe, and M. Gaboardi, “Privacy amplification by subsampling: Tight analyses via couplings and divergences,” *Advances in neural information processing systems*, vol. 31, 2018.
- [13] M. Blanton and C. Yuan, “Binary search in secure computation,” *Cryptology ePrint Archive*, 2021.
- [14] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai, “Lightweight techniques for private heavy hitters,” in *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 762–776, IEEE, 2021.
- [15] E. Boyle, N. Gilboa, and Y. Ishai, “Function secret sharing,” in *Annual international conference on the theory and applications of cryptographic techniques*, pp. 337–367, Springer, 2015.
- [16] L. Braun, M. Pancholi, R. Rachuri, and M. Simkin, “Ramen: Souper fast three-party computation for ram programs,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3284–3297, 2023.
- [17] J. Cartlidge, N. P. Smart, and Y. Talibi Alaoui, “Mpc joins the dark side,” in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, pp. 148–159, 2019.
- [18] T. H. Cormen, C. Lee, E. Lin, C. E. Leiserson, R. L. Rivest, and C. Stein, “Instructor’s manual,” 2002.
- [19] C. Thomas H, E. Charles, R. Ronald L, S. Clifford, *et al.*, “Introduction to algorithms third edition,” 2009.

- [20] B. H. Falk, R. Ostrovsky, M. Shtepel, and J. Zhang, “Gigadram: Breaking the billion address barrier,” *IACR Cryptol. ePrint Arch.*, vol. 2023, p. 1950, 2023.
- [21] N. Gilboa and Y. Ishai, “Distributed point functions and their applications,” in *Advances in Cryptology—EUROCRYPT 2014: 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings 33*, pp. 640–658, Springer, 2014.
- [22] O. Goldreich and R. Ostrovsky, “Software protection and simulation on oblivious rams,” *Journal of the ACM (JACM)*, vol. 43, no. 3, pp. 431–473, 1996.
- [23] Z. Jafargholi, K. G. Larsen, and M. Simkin, “Optimal oblivious priority queues,” in *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 2366–2383, SIAM, 2021.
- [24] S. Jarecki and B. Wei, “3pc oram with low latency, low bandwidth, and fast batch retrieval,” in *Applied Cryptography and Network Security: 16th International Conference, ACNS 2018, Leuven, Belgium, July 2-4, 2018, Proceedings 16*, pp. 360–378, Springer, 2018.
- [25] M. Keller and P. Scholl, “Efficient, oblivious data structures for mpc,” in *Advances in Cryptology—ASIACRYPT 2014: 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, ROC, December 7-11, 2014, Proceedings, Part II 20*, pp. 506–525, Springer, 2014.
- [26] S. Lu and R. Ostrovsky, “Distributed oblivious ram for secure two-party computation,” in *Theory of Cryptography Conference*, pp. 377–396, Springer, 2013.
- [27] S. Mazloom, B. Diamond, A. Polychroniadou, and T. Balch, “An efficient data-independent priority queue and its application to dark pools,” *Proceedings on Privacy Enhancing Technologies*, 2023.
- [28] M. N. Murty, V. S. Devi, M. N. Murty, and V. S. Devi, “Nearest neighbour based classifiers,” *Pattern Recognition: An Algorithmic Approach*, pp. 48–85, 2011.
- [29] R. L. Rivest, A. Shamir, and Y. Tauman, “How to leak a secret,” in *Advances in Cryptology—ASIACRYPT 2001: 7th International Conference on the Theory and Application of Cryptology and Information Security Gold Coast, Australia, December 9–13, 2001 Proceedings 7*, pp. 552–565, Springer, 2001.

- [30] S. Sasy and O. Ohrimenko, “Oblivious sampling algorithms for private data analysis,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [31] S. Servan-Schreiber, K. Hogan, and S. Devadas, “Adveil: A private targeted advertising ecosystem,” *Cryptology ePrint Archive*, 2021.
- [32] E. Shi, “Path oblivious heap: Optimal and practical oblivious priority queue,” in *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 842–858, IEEE, 2020.
- [33] E. Shi, T.-H. H. Chan, E. Stefanov, and M. Li, “Oblivious ram with $o((\log n)^3)$ worst-case cost,” in *International Conference on The Theory and Application of Cryptology and Information Security*, pp. 197–214, Springer, 2011.
- [34] K. Storrier, A. Vadapalli, A. Lyons, and R. Henry, “Grotto: Screaming fast $(2+1)$ -pc or $2n$ via $(2, 2)$ -dpfs,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2143–2157, 2023.
- [35] T. Toft, “Secure data structures based on multi-party computation,” in *Proceedings of the 30th annual ACM SIGACT-SIGOPS symposium on Principles of distributed computing*, pp. 291–292, 2011.
- [36] A. Vadapalli, F. Bayatbabolghani, and R. Henry, “You may also like... privacy: Recommendation systems meet pir,” *Proceedings on Privacy Enhancing Technologies*, 2021.
- [37] J. S. Vitter, “Random sampling with a reservoir,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 11, no. 1, pp. 37–57, 1985.
- [38] X. Wang, H. Chan, and E. Shi, “Circuit oram: On tightness of the goldreich-ostrovsky lower bound,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 850–861, 2015.
- [39] X. S. Wang, K. Nayak, C. Liu, T. H. Chan, E. Shi, E. Stefanov, and Y. Huang, “Oblivious data structures,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pp. 215–226, 2014.
- [40] S. Sasy, A. Vadapalli, and I. Goldberg, “Prac: Round-efficient 3-party mpc for dynamic data structures,” *Cryptology ePrint Archive*, 2023.
- [41] A. Vadapalli, *Distributed Point Functions and Their Applications to Secure Multi-Party Computation*. Indiana University, 2021.

- [42] S. Kiefer, J. Křetínskỳ, and A. Kučera, *Taming the Infinities of Concurrency: Essays Dedicated to Javier Esparza on the Occasion of His 60th Birthday*, vol. 14660. Springer Nature, 2024.
- [43] G. Cabodi, P. E. Camurati, M. Palena, and P. Pasini, “Hardware model checking algorithms and techniques,” *Algorithms*, vol. 17, no. 6, p. 253, 2024.
- [44] A. Canteaut and Y. Ishai, *Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II*, vol. 12106. Springer Nature, 2020.
- [45] J. Karhumäki, W. Plandowski, and F. Mignosi, “Automata, languages and programming,” 1997.
- [46] H. Handschuh and A. Lysyanskaya, *Advances in Cryptology–CRYPTO 2023: 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part I*, vol. 14081. Springer Nature, 2023.
- [47] D. Lin and K. Sako, “Public-key cryptography–pkc 2019,” in *22nd IACR international conference on practice and theory of public-key cryptography*, pp. 14–17, Springer, 2019.
- [48] D. P. Mehta and S. Sahni, *Handbook of data structures and applications*. Chapman and Hall/CRC, 2004.
- [49] L. Harn, M. Fuyou, and C.-C. Chang, “Verifiable secret sharing based on the chinese remainder theorem,” *Security and Communication Networks*, vol. 7, no. 6, pp. 950–957, 2014.
- [50] G. Zyskind *et al.*, *Secure Computation in Decentralized Systems*. PhD thesis, Massachusetts Institute of Technology, 2024.
- [51] A. Beimel and Y. Stahl, “Robust information-theoretic private information retrieval,” in *Security in Communication Networks: Third International Conference, SCN 2002 Amalfi, Italy, September 11–13, 2002 Revised Papers 3*, pp. 326–341, Springer, 2003.

- [52] M. Hirt, J. B. Nielsen, and B. Przydatek, “Asynchronous multi-party computation with quadratic communication,” in *International Colloquium on Automata, Languages, and Programming*, pp. 473–485, Springer, 2008.
- [53] Z. Wang, *Towards Secure Computation and Learning From Symmetric Private Information Retrieval*. PhD thesis, University of Maryland, College Park, 2023.
- [54] C. Deng, Y. Zhao, X. Tang, M. Gerstein, and A. Cohan, “Investigating data contamination in modern benchmarks for large language models,” *arXiv preprint arXiv:2311.09783*, 2023.
- [55] W. Dean, “Algorithms and the mathematical foundations of computer science,” *Gödel’s disjunction: The scope and limits of mathematical knowledge*, pp. 19–66, 2016.
- [56] L. Gasieniec, R. Martin, F. Wolter, and P. W. Wong, “Fundamentals of computation theory,” *Theor. Comput. Sci.*, vol. 590, p. 1, 2015.
- [57] J. A. Garay, *Public-Key Cryptography–PKC 2021: 24th IACR International Conference on Practice and Theory of Public Key Cryptography, Virtual Event, May 10–13, 2021, Proceedings, Part I*, vol. 12710. Springer Nature, 2021.
- [58] B. Ostrovsky, “Privacy-preserving dijkstra,” in *Annual International Cryptology Conference*, pp. 74–110, Springer, 2024.
- [59] D. Micciancio and T. Ristenpart, *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III*, vol. 12172. Springer Nature, 2020.
- [60] O. Matoba, T. Nomura, E. Perez-Cabre, M. S. Millan, and B. Javidi, “Optical techniques for information security,” *Proceedings of the IEEE*, vol. 97, no. 6, pp. 1128–1148, 2009.
- [61] R. Canetti, G. ROTHBLUM, and M. VARIA, *Theory of cryptography*. Springer, 2008.
- [62] B. G. Bakondi, A. Peter, M. Everts, P. Hartel, and W. Jonker, “Publicly verifiable private aggregation of time-series data,” in *2015 10th International Conference on Availability, Reliability and Security*, pp. 50–59, IEEE, 2015.

- [63] K. Mahalingam and K. Subramanian, “Product of parikh matrices and commutativity,” *International Journal of Foundations of Computer Science*, vol. 23, no. 01, pp. 207–223, 2012.
- [64] S. Khan, *Garbled Computation: Hiding Software, Data and Computed Values*. PhD thesis, Purdue University, 2024.
- [65] P. Sewell and J. Vitek, “Secure composition of insecure components,” in *Proceedings of the 12th IEEE Computer Security Foundations Workshop*, pp. 136–150, IEEE, 1999.
- [66] V. Dunjko, J. Fitzsimons, C. Portmann, and R. Renner, “Advances in cryptology—asiacrypt 2014,” 2014.
- [67] I. Ioannidis, A. Grama, and M. Atallah, “A secure protocol for computing dot-products in clustered and distributed environments,” in *Proceedings International Conference on Parallel Processing*, pp. 379–384, IEEE, 2002.
- [68] M. Hao, H. Li, H. Chen, P. Xing, and T. Zhang, “Fastsecnet: An efficient cryptographic framework for private neural network inference,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2569–2582, 2023.
- [69] M. Asad and S. Otoum, “Bppfl: a blockchain-based framework for privacy-preserving federated learning,” *Cluster Computing*, vol. 28, no. 2, p. 126, 2025.
- [70] K.-M. Chung, *Advances in Cryptology-ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VI.*, vol. 15489. Springer Nature, 2024.
- [71] E. Boyle and M. Mahmoody, *Theory of Cryptography*. Springer, 2025.
- [72] E. Qûşileviş, *Theory of Cryptography: 13th International Conference, TCC 2016-A, Tel Aviv, Israel, January 10-13, 2016: Proceedings*. Springer, 2016.
- [73] S. Moriai and H. Wang, *Advances in Cryptology-ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part III*, vol. 12493. Springer Nature, 2020.

- [74] A. Boldyreva and D. Micciancio, *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III*, vol. 11694. Springer, 2019.
- [75] R. Oracles, “Signcryption with quantum,” in *Provable Security: 12th International Conference, ProvSec 2018, Jeju, South Korea, October 25–28, 2018, Proceedings*, vol. 11192, p. 406, Springer, 2018.
- [76] H. E. Davis, *Securing the Standards: Bringing Cryptographic Security Proofs Closer to the Real World*. University of California, San Diego, 2023.